CS3400 — Linear Data Structures

Lecture Notes

---

1. Introduction to Linear Data Structures

A **linear data structure** stores elements in a sequential arrangement.

If a structure contains:

A = [a0 ,a1, a2,…, an−1]

then, conceptually, each element has a predecessor and/or successor, except for elements at the ends.

Examples include:

- Arrays
- Lists
- Linked lists
- Stacks
- Queues
- Deques

Linear structures differ mainly in **how elements are stored** and **which operations are efficient**.

For example:

Structure	**Main characteristic**
Array	Contiguous memory
Array-based List	Resizable array abstraction
Linked List	Nodes connected by pointers
Stack	LIFO access

Queue	FIFO access
Deque	Insertion/removal at both ends

A major distinction throughout this topic is:

> **The logical organization of a collection is not necessarily the same as its physical representation in memory.**

For example, a List ADT can be implemented using:

- an array,
- a dynamically allocated array,
- a singly linked list,
- or a doubly linked list.

The **ADT describes what the structure does**.

The **data structure describes how it is represented**.

---

2. Memory and Objects in C++

Before implementing dynamic data structures, we need to understand where objects live in memory.

2.1 Stack memory

Local variables normally have automatic storage duration.

```
void example()
{
   int x = 10;
   double y = 20.5;
}
```

When `example()` finishes, x and y cease to exist automatically.

This is one reason C++ programmers talk about **scope**. Remember local and global scope

---

2.2 Dynamic memory

Objects can also be created dynamically.

int *p = new int(10);

Here:

- p is a pointer.
- The pointer itself is a local variable.
- The `int` object was dynamically allocated.
- The dynamically allocated object remains alive until it is explicitly destroyed.

Therefore:

delete p;

is required.

After deletion, p should not be used to access the destroyed object.

A safer practice is:

delete p;
p = nullptr;

2.3 Why dynamic memory is important for data structures

Consider a linked list.

We do not normally know in advance how many nodes the list will contain.

We might begin with:

10

and later add:

10 -> 20

and later:

10 -> 20 -> 30 -> 40

The structure therefore needs to be capable of **growing and shrinking dynamically**.

Dynamic memory allows us to create nodes as required.

---

3. RAII — Resource Acquisition Is Initialization

RAII is one of the most important C++ concepts for writing reliable data structures.

RAII stands for:

> **Resource Acquisition Is Initialization**

The fundamental idea is:

> A resource should be associated with an object's lifetime.

When the object is created, it acquires the resource.

When the object is destroyed, it releases the resource.

Resources include:

- dynamically allocated memory,
- files,
- locks,
- sockets,
- database connections,
- operating-system handles.
- etc

For memory:

```
class Example
{
private:
   int *data;

public:
   Example()
   {
      data = new int(10);
   }

   ~Example()
   {
      delete data;
   }
};
```

When an `Example` object goes out of scope: Meaning example is called beyond this block

```
{
   Example e;
}
```

its destructor automatically executes.

Therefore:

delete data;

is automatically performed.

This is the fundamental idea behind RAII.

3.1 Why RAII matters for data structures

Suppose we create a linked list:

LinkedList list;

The programmer should not have to remember:

list.deleteAllNodes();

every time the list leaves scope.

Instead, the class should clean up its resources automatically through its destructor.

For example:

```
class LinkedList
{
private:
   Node* head;

public:
   ~LinkedList()
   {
      clear();
   }
};
```

This gives the class ownership of its dynamically allocated nodes.

3.2 Modern C++ and RAII

Modern C++ generally prefers:

std::unique_ptr

and

std::shared_ptr

over manually managing ownership with raw pointers.

However, for **CS3400**, students should understand raw pointers because implementing linked structures requires understanding what is happening underneath.

A good progression is:

Raw pointers
   ↓
Dynamic memory
   ↓

Destructors
↓
RAII
↓
Smart pointers

---

4. Memory Leaks

A **memory leak** occurs when dynamically allocated memory is no longer reachable but has not been released.

Example:

```
void example()
{
   int* p = new int(10);

   p = nullptr;
}
```

What happened?

The dynamically allocated integer still exists, but we lost the address needed to delete it.

Therefore:

```
delete p;
```

can no longer be performed.

The memory has leaked.

4.1 Another common leak

```
int* p = new int(10);

p = new int(20);

delete p;
```

The first allocation is leaked.

The pointer was changed to point to the second allocation before the first allocation was released.

4.2 Memory leak in a linked list

Suppose:

```
head
 ↓
[10] -> [20] -> [30] -> nullptr
```

If we simply do:

```
head = nullptr;
```

we lose access to all three nodes.

The nodes still occupy memory.

Therefore, the correct destruction process is:

```
while (head != nullptr)
{
   Node* temp = head;
   head = head->next;
   delete temp;
}
```

This is an important pattern that students should understand.

---

5. Dangling Pointers

A **dangling pointer** points to an object that has already been destroyed.

Example:

```
int* p = new int(10);

delete p;
```

```
cout << *p;
```

p still contains an address, but the object at that address no longer exists.

Therefore, dereferencing p produces undefined behavior.

Better:

```
delete p;
p = nullptr;
```

---

6. Shallow Copy and Deep Copy

This is particularly important when implementing linked structures.

Consider:

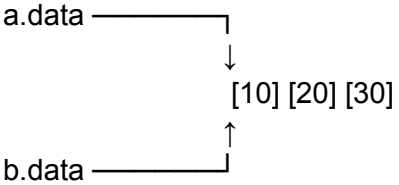
```
class Array
{
private:
    int* data;
};
```

Suppose:

```
Array a;
Array b = a;
```

What should happen?

If the compiler performs member-wise copying, we could have:



Both objects contain pointers to the **same memory**.

This is called a **shallow copy**.

6.1 Why shallow copying is dangerous

Suppose:

Array a;
Array b = a;

Then:

delete[] a.data;

The memory is destroyed.

But:

b.data

still points to the destroyed memory.

Now b contains a dangling pointer.

Even worse, when b is destroyed, its destructor might attempt:

delete[] b.data;

resulting in a **double deletion**.

6.2 Deep copy

A **deep copy** creates an independent copy of the dynamically allocated resource.

Conceptually:

a.data ————> [10][20][30]

b.data ————> [10][20][30]

The two objects have separate arrays.

Changing one does not affect the other.

6.3 Example

```
class DynamicArray
{
private:
    int* data;
    int size;

public:
    DynamicArray(int n)
    {
        size = n;
        data = new int[size];
    }

    ~DynamicArray()
    {
        delete[] data;
    }

    DynamicArray(const DynamicArray& other)
    {
        size = other.size;
        data = new int[size];

        for (int i = 0; i < size; ++i)
        {
            data[i] = other.data[i];
        }
    }
};
```

The copy constructor performs a deep copy.

---

7. The Rule of Three

If a class manages a resource manually, three special member functions often need to be considered:

1. Destructor
2. Copy constructor
3. Copy assignment operator

This is called the **Rule of Three**.

For example:

```
class DynamicArray
{
private:
   int* data;
   int size;

public:

   ~DynamicArray();

   DynamicArray(const DynamicArray& other);

   DynamicArray& operator=(const DynamicArray& other);
};
```

Why all three?

Because the class owns dynamically allocated memory.

7.1 Copy constructor

Used when creating a new object from an existing object:

```
DynamicArray b = a;
```

or:

```
DynamicArray b(a);
```

7.2 Copy assignment

Used when both objects already exist:

b = a;

The implementation must first release b's old resource and then copy a's resource.

7.3 Destructor

Used when the object dies:

```
~DynamicArray()
{
   delete[] data;
}
```

7.4 Rule of Five

Modern C++ extends this idea.

The **Rule of Five** includes:

1. Destructor
2. Copy constructor
3. Copy assignment operator
4. Move constructor
5. Move assignment operator

Move semantics allow resources to be transferred rather than copied.

For example:

Object A owns memory
↓ move
Object B owns memory

A no longer owns it

This becomes particularly important when working with efficient C++ containers.

For this section of CS3400, however, students should first become comfortable with the Rule of Three before going deeply into move semantics.

---

8. Arrays

An array stores elements in **contiguous memory locations**.

For:

int numbers[5];

conceptually:

Address	1000	1004	1008	1012	1016
	↓	↓	↓	↓	↓
value	[10]	[20]	[30]	[40]	[50]
Index	0	1	2	3	4

Assuming an `int` occupies four bytes. We did this in class

8.1 Characteristics of arrays

Arrays have several important characteristics:

1. Fixed size

For a built-in array:

int numbers[10];

the number of elements is fixed.

2. Contiguous memory

Elements are stored next to one another.

3. Constant-time indexing

Accessing:

numbers[7]

is approximately:

O(1)

because the address can be calculated directly.

8.2 Array address calculation

If:

- base address = B
- element index = `i`
- element size = s

then:

Address(A[i])=B+i×s

This is why array indexing is efficient.

---

9. Static and Dynamic Arrays

Static array

int values[100];

The size is fixed.

If only 10 positions are used, the remaining capacity still exists.

Dynamic array

We can allocate an array at runtime:

int n;

cin >> n;

int* values = new int[n];

Now the size is determined at runtime.

But we must eventually perform:

delete[] values;

9.1 The problem with dynamic arrays

Suppose:

int* data = new int[5];

The array has capacity 5.

What happens when we need a sixth element?

We cannot simply extend the allocation.

Instead, we typically:

1. Allocate a larger array.
2. Copy the existing elements.
3. Delete the old array.
4. Make the pointer point to the new array.

For example:

Old: [10][20][30][40][50]

↓ allocate

New: [10][20][30][40][50][ ][ ][ ][ ][ ]

This is the basis of an **array-based list**.

---

10. List ADT

A **List ADT** represents an ordered collection of elements.

Typical operations include:

insert(position, element)
remove(position)
get(position)
set(position, element)
size()
isEmpty()
clear()
search(element)

The List ADT does not dictate whether the list is implemented using:

- an array,
- a dynamic array,
- a linked list.

10.1 Array-based list

Suppose:

[10][20][30][40]

and we insert 25 at position 2.

We need:

[10][20][25][30][40]

The elements after position 2 must move.

Before:

10 20 30 40
↑
insert

After:

10 20 25 30 40

Therefore insertion can be expensive.

10.2 Array-based list operations

Operation	Complexity
Access by index	O(1)
Search	O(n)
Insert at beginning	O(n)
Insert at middle	O(n)
Insert at end	O(1) amortized
Remove at beginning	O(n)
Remove at middle	O(n)
Remove at end	O(1)

The phrase **amortized O(1)** is important.

If an array needs to resize, one particular insertion may cost O(n).

But over many insertions, the average cost can be O(1), assuming a suitable growth strategy such as doubling capacity.

---

11. Linked Lists

A linked list stores elements in **nodes**.

A node contains:

data
pointer to another node

For a singly linked list:

```
struct Node
{
   int data;
   Node* next;
};
```

A list might look like:

```
head
 ↓
[10 | •]---> [20 | •]---> [30 | nullptr]
```

Unlike an array, nodes do not have to occupy contiguous memory.

11.1 Dynamic linked structures

A node can be created dynamically:

```
Node* node = new Node;
```

and initialized:

```
node->data = 10;
node->next = nullptr;
```

We can also write:

```
Node* node = new Node{10, nullptr};
```

---

12. Singly Linked Lists

A singly linked list has one link per node:

[data | next]

Example:

head
↓
[10] → [20] → [30] → [40] → nullptr

The final node points to:

nullptr

12.1 Traversal

To traverse:

```
Node* current = head;

while (current != nullptr)
{
    cout << current->data << endl;
    current = current->next;
}
```

The critical operation is:

current = current->next;

This moves from one node to the next.

Traversal complexity: O(n)

12.2 Searching

bool contains(int value)

```
{
    Node* current = head;

    while (current != nullptr)
    {
        if (current->data == value)
            return true;

        current = current->next;
    }

    return false;
}
```

Worst-case complexity: O(n)

because we may need to inspect every node.

---

13. Insertion in a Singly Linked List

Insert at beginning

Suppose:

head
↓
10 → 20 → 30

To insert 5:

```
Node* newNode = new Node{5, head};
head = newNode;
```

Result:

head
↓

5 → 10 → 20 → 30

Complexity: O(1)

13.1 Insert after a node

Suppose:

10 → 20 → 30

We want:

10 → 20 → 25 → 30

The important pointer manipulation is:

```
newNode->next = current->next;
current->next = newNode;
```

The order matters.

If we incorrectly overwrite `current->next` first, we may lose access to the rest of the list.

---

14. Deletion

Suppose:

10 → 20 → 30

We want to delete `20`.

We need the predecessor:

10
↓

20
↓
30

We change:

10 → 30

and then destroy the old node:

delete nodeToDelete;

This illustrates an important principle:

> **Removing a node from a linked structure has two separate responsibilities: disconnect the node and release its memory.**

Students often perform the first and forget the second.

That creates a memory leak.

---

15. Doubly Linked Lists

A doubly linked node contains two links:

```
struct Node
{
   int data;
   Node* previous;
   Node* next;
};
```

Conceptually:

nullptr ← [10] ⇄ [20] ⇄ [30] → nullptr

Each node can move:

- forward,
- backward.

15.1 Advantages

Doubly linked lists make certain operations easier.

For example, if we already have a pointer to a node, we can access:

node->previous

without searching for its predecessor.

15.2 Disadvantages

Each node requires additional memory.

A singly linked node:

data + one pointer

A doubly linked node:

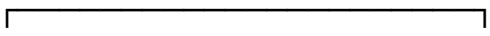
data + two pointers

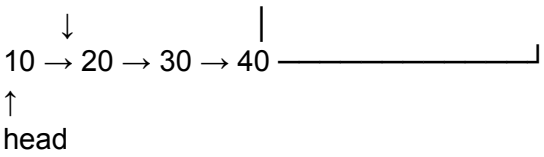
Insertion and deletion also require more pointer updates.

---

16. Circular Linked Lists

In a circular linked list, the last node points back to the first node.





There is no:

nullptr

at the end.

Instead:

last->next == head

16.1 Why use circular lists?

They are useful for structures where processing repeatedly cycles through elements.

Examples:

- Round-robin scheduling
- Repeated turn-taking
- Circular buffers
- Multiplayer turn systems

16.2 Traversing a circular list

We cannot write:

while (current != nullptr)

because `current` will never become `nullptr`.

Instead:

```
Node* current = head;

do
{
   cout << current->data << endl;
   current = current->next;
}
while (current != head);
```

This is a good example of how the **representation determines the algorithm**.

---

17. Linked List Complexity

Operation	Singly Linked List
Access by index	O(n)
Search	O(n)
Insert at beginning	O(1)
Remove beginning	O(1)
Insert after known node	O(1)
Remove after known predecessor	O(1)
Insert at end with tail	O(1)
Search for insertion position	O(n)

Notice the difference between:

> **Finding the location**

and

> **Performing the operation once the location is known.**

For example, inserting after a known node is O(1), but finding that node may require O(n).

---

18. Comparing Arrays and Linked Lists

Feature	Array	Linked List
Memory	Contiguous	Non-contiguous
Random access	O(1)	O(n)
Insert beginning	O(n)	O(1)
Delete beginning	O(n)	O(1)
Memory overhead	Low	Higher
Resizing	Required for dynamic arrays	Naturally dynamic
Cache locality	Excellent	Generally poorer
Pointer manipulation	Minimal	Significant

An important lesson is:

> **Asymptotic complexity is not the only factor affecting performance.**

Arrays often perform very well because their contiguous memory gives them good **cache locality**.

What is this cache locality

---

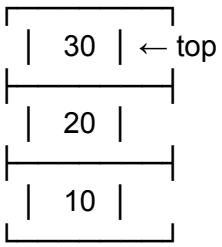
19. Stacks

A **Stack ADT** follows the:

> **LIFO — Last In, First Out**

principle.

Imagine a stack of plates.



The last element inserted is the first one removed.

19.1 Fundamental stack operations

push

Adds an element.

push(40)

pop

Removes the top element.

top / peek

Returns the top element without removing it.

isEmpty

Determines whether the stack contains elements.

---

20. Array-Based Stack

Suppose:

```
class Stack
{
private:
    int data[100];
    int topIndex;
};
```

We can maintain:

topIndex

representing the top position.

For example:

```
index:    0   1   2
          ↓   ↓   ↓
data:    10  20  30
                  ↑
                 top
```

20.1 Push

```
data[++topIndex] = value;
```

Complexity: O(1)

20.2 Pop

```
return data[topIndex--];
```

Complexity: O(1)

---

21. Linked Stack

A stack can also be implemented using a linked list.

top
↓
[30] → [20] → [10] → nullptr

Push:

new node
  ↓
[40] → [30] → [20] → [10]
↑
top

Again:

push=O(1)

and:

pop=O(1)

---

22. Stack Applications

Stacks are used extensively in computing.

1. Function calls

When a function is called, information about the call is maintained on the call stack.

2. Recursion

Each recursive call creates another stack frame.

3. Expression evaluation

For example: (10 + 20) * 5

4. Parentheses matching

{ [( )] }

5. Undo operations

A sequence of actions can be stored on a stack.

6. Depth-first search

DFS naturally uses stack behavior.

7. Backtracking

Examples:

- maze solving,
- puzzle solving,
- state-space exploration.

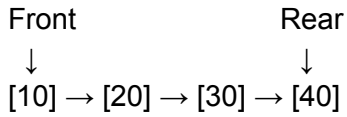
---

23. Queues

A **Queue ADT** follows:

FIFO — First In, First Out

Think of people waiting in a line.



The first person to enter the queue is the first person served.

23.1 Queue operations

enqueue

Adds an element at the rear.

dequeue

Removes an element from the front.

front

Returns the front element without removing it.

rear

Returns the last element.

isEmpty

Tests whether the queue is empty.

---

24. Naive Array Queue

Suppose:

[10][20][30][40]
↑
front

If we remove `10`, a naive implementation could shift everything:

[20][30][40]

But this requires: O(n)

work.

That is inefficient.

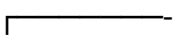
A better solution is a **circular queue**.

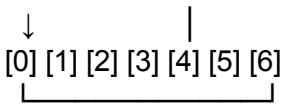
---

25. Circular Queue

A circular queue treats the array as if its end connects back to its beginning.

Conceptually:





If rear reaches the final array position, it wraps around:

rear=(rear+1) capacity

For example, if capacity is 5:

(4 + 1) % 5 = 0

So position 4 is followed by position 0.

25.1 Why circular queues?

Consider:

capacity = 5

[ ][ ][ ][ ][ ]

After enqueueing:

[10][20][30][40][50]

and dequeuing three:

[ ][ ][ ][40][50]

There is free space at the beginning.

A circular queue can reuse it:

[60][70][80][40][50]

without moving the existing elements.

This allows enqueue and dequeue to remain: O(1)

---

26. Circular Queue Full and Empty Conditions

One common implementation maintains:

front
rear
size

Then:

empty = (size == 0)

and:

full = (size == capacity)

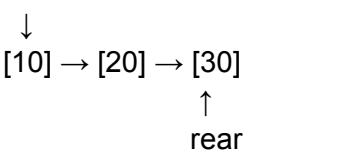
This is often easier to reason about than implementations that leave one position deliberately unused.

---

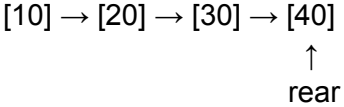
27. Linked Queue

A queue can also be implemented using linked nodes.

Maintain:



Enqueue at the rear:



Dequeue from the front:

front
↓
[20] → [30] → [40]

With both front and rear pointers:

enqueue=O(1) dequeue=O(1)

---

28. Double-Ended Queues — Deques

A **deque** is a:

Double-Ended Queue

It allows insertion and removal at both ends.

Conceptually:

Front Rear
↓ ↓
[10] ⇄ [20] ⇄ [30] ⇄ [40]

Possible operations:

push_front()
push_back()
pop_front()
pop_back()

All four can be designed to operate in: O(1)

with an appropriate implementation.

28.1 Applications of deques

Deques are useful for:

- task scheduling,
- sliding-window algorithms,
- browser history structures,
- caching,
- palindrome checking,
- algorithms requiring access to both ends.

---

29. Queue Applications

Queues appear throughout computing.

Operating systems

Processes may wait for CPU time.

Networking

Packets may wait for transmission.

Printers

Print jobs wait to be processed.

Web servers

Requests may be queued.

Breadth-first search

BFS uses queue behavior.

Simulation

Queues can model:

- customers,
- vehicles,
- patients,
- jobs,
- network packets.

---

30. Memory Management in Linked Structures

This is one of the most important implementation issues.

Suppose:

head
↓
A → B → C → nullptr

If we remove B:

A → C → nullptr

we must also destroy B:

delete B;

Otherwise:

A → C

B still exists somewhere in memory

but no pointer reaches it.

That is a **memory leak**.

---

31. Destructor for a Linked List

A linked list owning its nodes should release them.

```
~LinkedList()
{
   Node* current = head;

   while (current != nullptr)
   {
      Node* next = current->next;
      delete current;
      current = next;
   }
}
```

Notice the ordering:

```
Node* next = current->next;
delete current;
current = next;
```

We save `next` **before** deleting `current`.

If we did:

```
delete current;
current = current->next;
```

we would access an object after it has been destroyed.

That is undefined behavior.

---

32. Linked List Copying

Suppose:

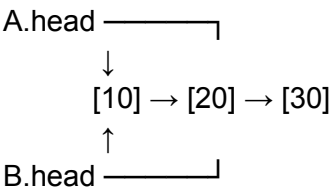
List A:

head
↓
[10] → [20] → [30]

We write:

List B = A;

A shallow copy could result in:



Now both lists think they own the same nodes.

This is a serious design error.

32.1 Correct deep copy

We want:

A.head → [10] → [20] → [30]

B.head → [10] → [20] → [30]

but these must be **different nodes**.

Conceptually:

```
List(const List& other)
{
   // Create new nodes containing the same values.
}
```

This is another application of the Rule of Three.

---

33. Ownership

A crucial concept when implementing data structures is:

Who owns the dynamically allocated memory?

Suppose:

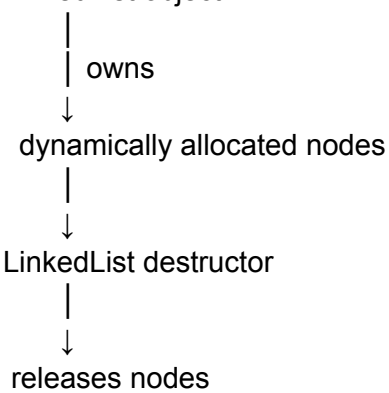
Node* p = new Node{10, nullptr};

Someone must be responsible for eventually doing:

delete p;

If a `LinkedList` creates and manages its nodes, the list should normally be considered the **owner** of those nodes.

This leads naturally to RAII:



---

34. Common Memory Errors

Students implementing linked structures should be able to identify these.

Memory leak

Memory allocated but never released.

Dangling pointer

Pointer refers to an object that has already been destroyed.

Double deletion

delete p;
delete p;

The same resource is released twice.

Use-after-free

delete p;
cout << p->data;

Accessing an object after it has been destroyed.

Lost pointer

head = head->next;

performed without first preserving the old node when that node still needs to be deleted.

---

35. Summary of Linear Structures

Structure	Access	Insert	Remove	Principle
Array	O(1)	O(n)	O(n)	Contiguous storage
Linked List	O(n)	O(1)*	O(1)*	Nodes + links
Stack	O(1) top	O(1)	O(1)	LIFO
Queue	O(1) ends	O(1)	O(1)	FIFO
Deque	O(1) ends	O(1)	O(1)	Both ends

Assumes the relevant position/node is already known.
